Sentinel

AI-Driven Integrated Surveillance System

for Threat Recognition and Emergency Response

Project documentation: architecture, AI models, implementation, testing, installation and operation

Version	1.0
Date	October 2026
Stack	Python FastAPI · PyTorch · Node.js Express · MongoDB · React

Contents

1. Introduction 3

2. System architecture 4

3. AI models 5

4. Detection pipeline 6

5. Threat scoring and alerting 7

6. Backend and database 8

7. Admin panel 9

8. Testing and results 11

9. Installation and running 12

10. Limitations and future work 13

1. Introduction

Security teams cannot watch every camera feed continuously. Weapons, fights, fires and injuries are often noticed only after the critical first minutes. This project builds a complete system that watches a live camera feed with artificial intelligence, recognises dangerous situations as they happen, and alerts the people responsible within seconds, with photographic evidence.

1.1 Objectives

- Analyse every live camera frame with pretrained deep-learning models. No model training or dataset preparation is needed.
- Detect **fights / physical violence**, **guns**, **knives**, **blood**, **fire**, **smoke** and **suspicious activity** (a person holding a weapon, or a fight involving a weapon).
- Turn the detections into a single **threat score from 0 to 100** and a level: Safe, Medium Danger or High Danger.
- On a threat: save an annotated screenshot, store the incident in a database, send an **email alert** with the screenshot, and show it **live** on an admin panel.
- Provide a secure admin panel with a live view, full detection history, statistics and settings.
- Run on an ordinary Windows laptop with its built-in webcam.

1.2 Scope

The system is a working prototype for one camera, the laptop webcam. Any USB or IP camera (RTSP), or a video file, can be used instead through one configuration value. All AI models are public pretrained checkpoints downloaded from Hugging Face and Ultralytics.

2. System architecture

The system has three independent applications that talk over HTTP. Each can be restarted without losing data, and the AI service keeps working even if the backend is temporarily down.

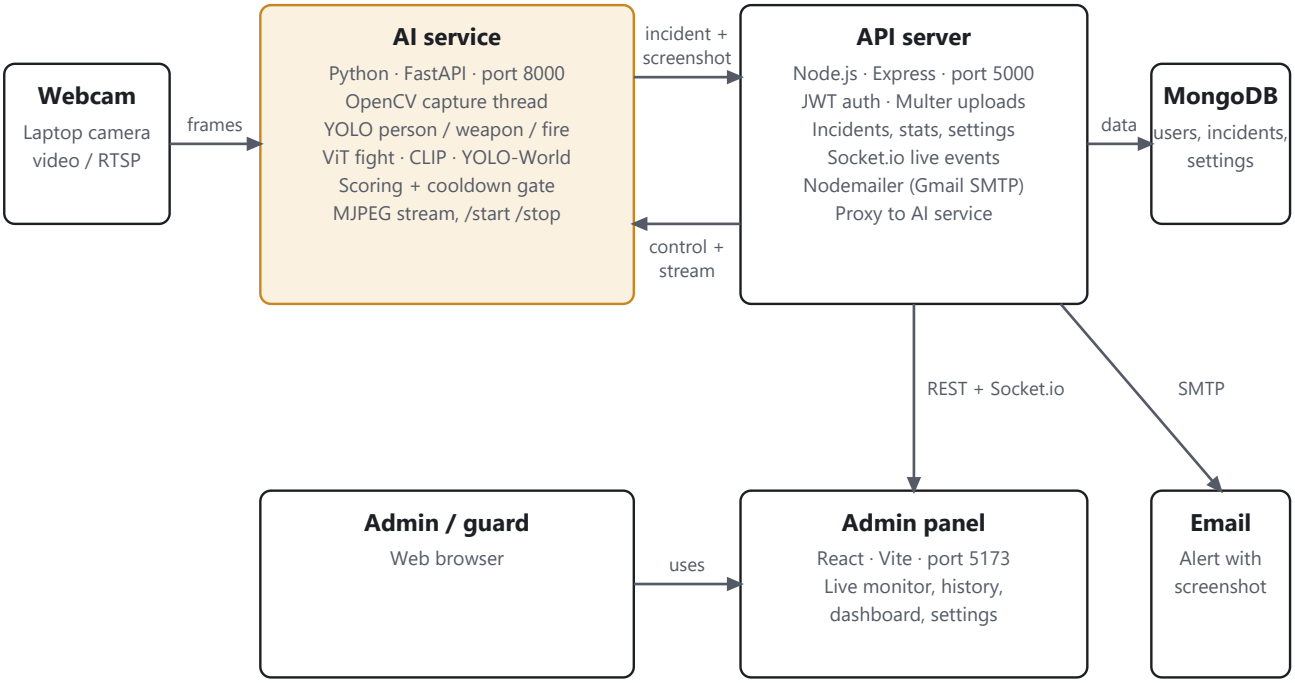


Figure 1: Components and data flow.

Component	Technology	Responsibility
AI service	Python 3.10+, FastAPI, Uvicorn, OpenCV, PyTorch, Ultralytics, transformers, timm	Captures camera frames, runs all detectors, computes the threat score, draws annotations, streams MJPEG video and sends incidents to the backend
API server	Node.js, Express 5, Mongoose, JWT, bcrypt, Multer, Nodemailer, Socket.io	Authentication, incident storage, screenshots, email alerts, statistics, settings, live push to browsers, proxy for camera control and the video stream
Database	MongoDB (local or Atlas)	Users, incidents and settings
Admin panel	React 18, Vite, React Router, Axios, Socket.io client, Recharts, Lucide icons	Login, live monitor, detection history, dashboard, settings

2.1 Communication

- **AI service to API:** `POST /api/incidents/ingest` (multipart form with the screenshot), protected by a shared secret in the `x-api-key` header.
- **API to AI service:** start/stop/status calls and the MJPEG stream are proxied, so the browser only ever talks to the API.
- **API to browser:** REST with a JWT bearer token, plus Socket.io events (`new_incident`, `incident_updated`, `incident_deleted`) for instant updates.
- **API to email:** Gmail SMTP over SSL (port 465) using a Gmail App Password.

3. AI models

No model is trained in this project. For every detector, a ranked list of public candidates is kept in `ai-service/app/config.py`. At start-up the service checks with `huggingface_hub` that each candidate repository exists and contains its weights, downloads the first usable one, and falls back to the next candidate. Zero-shot YOLO-World is the final fallback. Candidates were ranked by the accuracy or mAP50 reported on their model cards.

Detector	Model selected	Reported accuracy	Role
Person	Ultralytics YOLOv8n (COCO)	COCO pretrained	Counts people; links weapons to people; COCO 'knife' used as an extra weapon signal
Weapons	cosgun99/gun-knife-yolo11n	mAP50 0.964 (gun 0.984, knife 0.944)	Gun and knife boxes. Best among all weapon checkpoints found
Weapons fallback	Subh775/Threat-Detection-YOLOv8n, then YOLO-World v2-X	mAP50 0.811 / zero-shot	Used only if the primary model is unavailable
Fire & smoke	rabahdev/fire-smoke-yolov8n (D-Fire)	mAP50 0.754 on 4,306 test images	Fire and smoke boxes
Fight / violence	jaranohaal/vit-base-violence-detection (ViT-B/16)	98.8% test accuracy (Real Life Violence Situations)	Frame-level violent / non-violent probability
Blood	YOLO-World v2-S 'blood' + openai/clip-vit-large-patch14	Zero-shot	No reliable dedicated blood detector exists, so two zero-shot models vote

3.1 Findings during model verification

Every model was loaded and run on test images before integration. Two problems were found and fixed:

- **The violence model ships timm-format weights** (`blocks.N.attn.qkv`), although its model card shows the transformers class. Loading it that way silently produces *random* weights. The service detects the format and loads the weights into the matching timm ViT-B/16 in strict mode. Any transformers model that reports missing weights is rejected.
- **Library change in transformers 5:** CLIP feature functions now return an output object instead of a tensor. The code supports both versions.

3.2 False-alarm safeguards

Testing on the live webcam and on public images showed false positives that a demo or a client would notice. Because CLIP is already loaded for blood, one CLIP pass per sample scores the whole scene against prompt groups (blood, fire, smoke, fight, and a broad 'normal' group) and acts as a second opinion:

Observed problem	Safeguard
Fire model reported 'smoke' on plain grey walls (0.57-0.74)	Fire/smoke must be seen in 3 of the last 5 frames **and** CLIP fire+smoke probability must be at least 0.40
Violence model gave 0.95 'violent' for one calm person in a dark room	A fight needs 2 or more people; CLIP vetoes it if its fight probability is below 0.15 (real fights scored about 1.0)
CLIP alone called knife product photos 'blood' (0.98)	Broad negative prompts (products, tools, faces, dark rooms, drawings) added to the normal group
Single-frame 'gun' flicker (0.45) on a hand near the face in low light	A weapon must be detected in 2 of the last 3 processed frames

4. Detection pipeline

The camera, the detectors and the video stream run in separate threads, so a slow model never makes the video stutter. The stream always shows the newest frame with the latest results drawn on it.

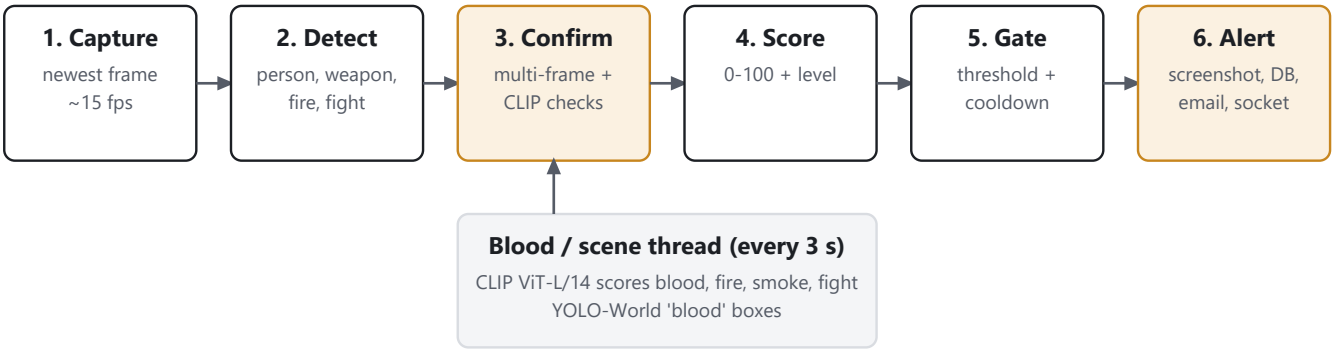


Figure 2: Processing steps for each analysed frame.

Detector	When it runs	Reported when
Person (YOLOv8n)	Every detection cycle, 640 px	Confidence at least 0.40
Weapons (YOLO11n + COCO knife)	Every cycle	Confidence at least 0.45 (COCO knife 0.40), in 2 of the last 3 cycles
Fire / smoke (YOLOv8n D-Fire)	Every cycle	Confidence at least 0.40, in 3 of the last 5 cycles, and CLIP agrees
Fight (ViT-B/16)	Every 3rd frame, only when 2+ people are visible	Mean of the last 8 probabilities at least 0.70, 2+ people, no CLIP veto
Blood (YOLO-World + CLIP)	Separate thread, every 3 s	(YOLO-World 'blood' and CLIP at least 0.50) or CLIP at least 0.80
Suspicious activity	Derived	A weapon box centre inside a person box, or a fight with a weapon

Each analysed frame is annotated with bounding boxes, labels, confidences, and a header bar showing the score and level in green (Safe), amber (Medium) or red (High). The screenshot attached to an incident is exactly this annotated frame.



Figure 3: Annotated incident screenshot produced by the system (gun detected, score 90, High Danger).

5. Threat scoring and alerting

Threat	Score
Fire	100
Gun	90
Knife	70
Fight	60 + 30 × (p − 0.70) / 0.30, where p is the smoothed fight probability (60 to 90)
Blood	50
Smoke (without fire)	45
Person holding a weapon	+10 added to the weapon score
Several threat types	final = min(100, highest score + 10 for each additional threat type)

Score	Level	Meaning
0 - 39	Safe	No action
40 - 69	Medium Danger	Incident raised (default alert threshold 40)
70 - 100	High Danger	Incident raised; red alert in the panel and the email subject

5.1 Worked examples (verified by test)

Situation	Calculation	Result
Gun visible	90	90 High
Person holding a gun	90 + 10	100 High
Knife and blood, person holding the knife	max(70 + 10, 50) + 10	90 High
Fight with smoothed p = 0.85	60 + 30 × 0.15 / 0.30	75 High
Smoke only	45	45 Medium
Fire and smoke	Smoke counts as part of the fire	100 High

5.2 Cooldown

After an alert, the same threat type cannot trigger a new alert for 60 seconds, so one event does not produce dozens of emails. A **different** threat type triggers immediately. The threshold and the cooldown are edited on the Settings page; the AI service picks up changes within 20 seconds without a restart.

5.3 What happens on an alert

- The annotated frame is saved as a JPEG (a local copy is kept in `ai-service/captures`).
- The incident is queued and posted to the API. If the API is down, it stays in an in-memory queue (up to 200 incidents) and is retried every 5 seconds; the AI service never crashes.
- The API stores the screenshot and the incident and immediately pushes `new_incident` to all logged-in browsers (toast + alert sound).
- The email is sent in the background, e.g. subject: *[HIGH DANGER] Gun detected - Score 90 - 13 Oct 2026 01:12 AM*, with threats, score, level, time (Asia/Kolkata), camera, a link to the incident, and the screenshot inline and attached. The result (sent or the error) is recorded on the incident.

6. Backend and database

6.1 Data model (MongoDB)

Collection	Fields
users	name, email (unique), passwordHash (bcrypt, 12 rounds), role ('admin'), timestamps
incidents	threats [String], primaryThreat, score (0-100), level, confidences (Map of Number), screenshotUrl, cameraName, emailSent, emailError, status (new / acknowledged / resolved / false_alarm), notes, detectedAt, createdAt, updatedAt
settings	alertEmails [String], cooldownSeconds, alertThreshold, emailEnabled (single document)

6.2 REST API

Method & path	Auth	Purpose
POST /api/auth/login	-	Email + password, returns a JWT valid for 8 hours
GET /api/auth/me	JWT	Current admin
POST /api/incidents/ingest	API key	Used by the AI service: screenshot upload, save, email, live push
GET /api/incidents	JWT	Pagination, filters (threat, level, status, date range, search), newest first
GET /api/incidents/export	JWT	CSV export with the same filters
GET / PATCH / DELETE /api/incidents/:id	JWT	Detail; update status and notes; delete with its screenshot
GET /api/stats?days=7|30	JWT	Totals, today, high danger, by threat, by level, per day
GET / PUT /api/settings	JWT	Alert emails, cooldown, threshold, email on/off
POST /api/settings/test-email	JWT	Sends a test email
/api/camera/start, stop, status, video_feed	JWT	Proxy to the AI service (camera control and MJPEG stream)
GET /uploads/screenshots/*	-	Screenshot files

6.3 Security

- Passwords hashed with bcrypt; JWT signed with a random secret generated during setup; 8-hour expiry with automatic logout in the panel.
- The ingest endpoint uses a separate shared API key, compared in constant time.
- Socket.io connections require a valid JWT, so only logged-in admins receive live alerts.
- Secrets live in `.env` files, which are excluded from Git.
- Uploads are limited to images up to 10 MB; inputs are validated (status values, email addresses, threshold and cooldown ranges).

7. Admin panel

A dark, professional interface (near-black background, one amber accent, red and green reserved for danger and safe states, Inter font, Lucide icons). It is responsive down to phone width and keyboard accessible. Levels are always shown with an icon and text, never colour alone.

Page	Features
Login	Email and password; session-expired notice; protected routes
Live monitor	Annotated MJPEG video; Start/Stop camera; live score gauge and level badge; active threats with confidence; stream FPS, detection rate, people count, fight probability; loaded models; recent incidents; real-time toast and sound for new incidents; Test alert button
Detection history	Table with screenshot thumbnail, time, threats, score, level, status, email status; search, filters, date range, pagination; CSV export; live refresh
Incident detail	Full screenshot, model confidences, email result, status update, notes, delete
Dashboard	Total, today, high danger, awaiting review; incidents per day (7 or 30 days); by threat type; by danger level
Settings	Alert recipients, email on/off, send test email, alert threshold, cooldown

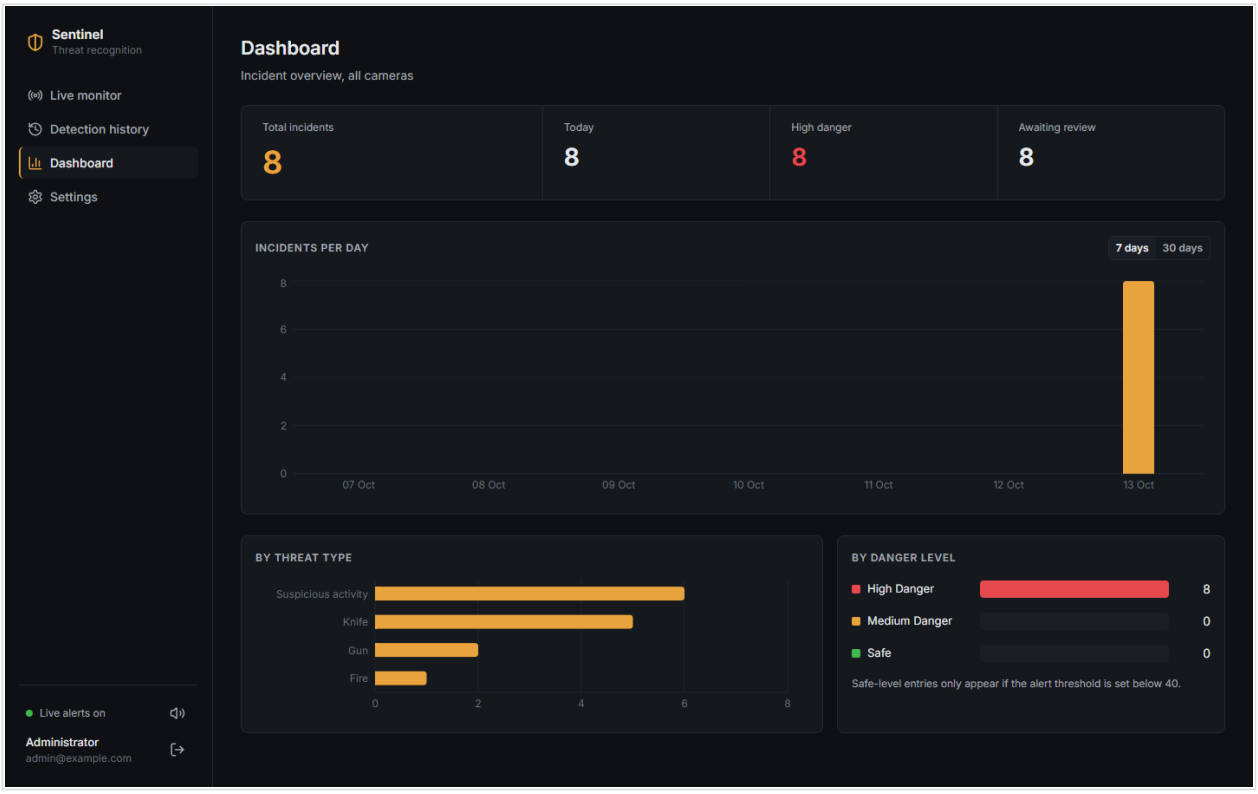


Figure 4: Dashboard.

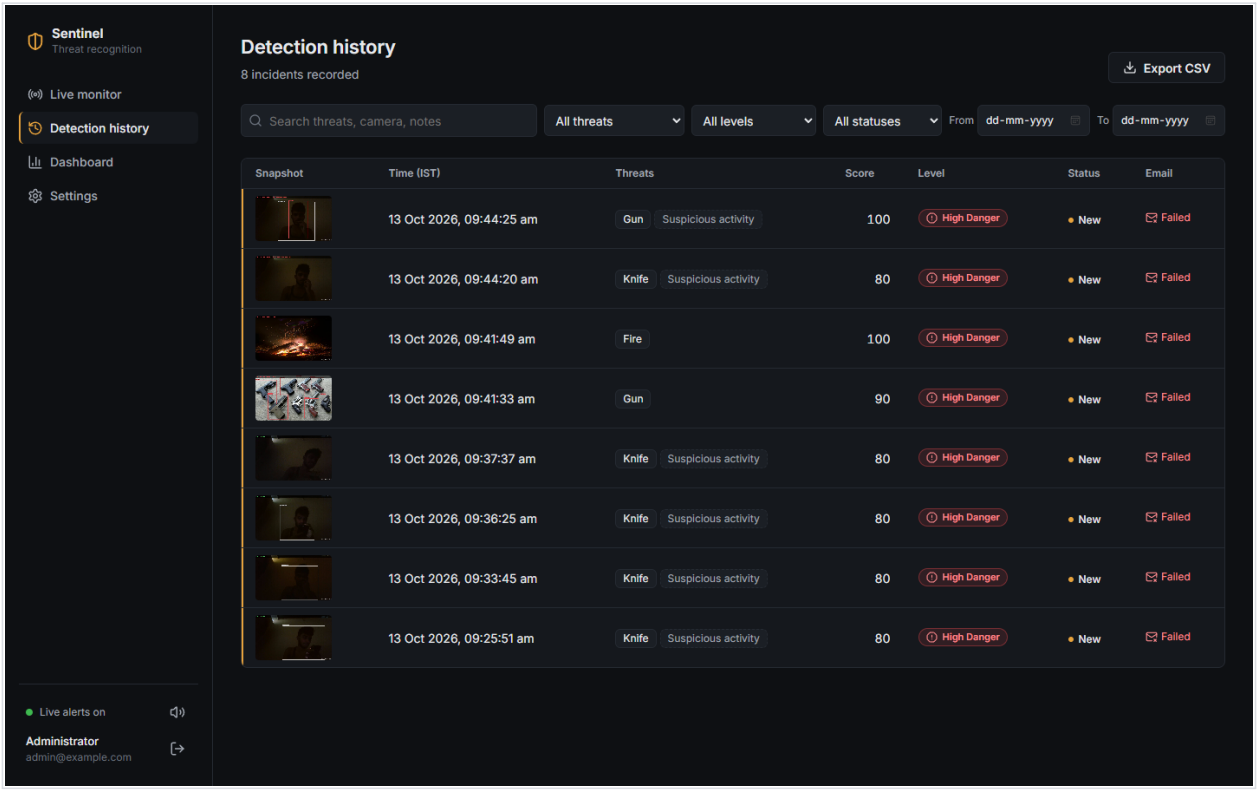


Figure 5: Detection history.

8. Testing and results

Testing covered every layer, using the real webcam, public test images and a looping test video fed through the live pipeline.

Test	Method	Result
API functions	Automated script: login (valid/invalid), protected routes, wrong API key, ingest with screenshot, filters, status update, stats, settings, CSV, delete	All passed
Weapon detection	Photo of handguns	9 guns detected (0.60-0.86), score 90 High
Fire detection	Campfire photos; candle and orange-lit room as look-alikes	Both fires detected (score 100); no false alarm on the look-alikes
Blood	Wound photos vs. knife product photos	Wound flagged; knife photos no longer flagged after the prompt fix
Full pipeline	Looping video (calm room, then guns, then fire) as the camera source	Exactly one gun incident (90) and one fire incident (100); cooldown suppressed repeats; calm frames scored 0
False alarms on the live webcam	60 seconds of the real laptop camera, person in view	0 incidents, maximum score 0
Scoring rules	12 scripted cases (combinations, bonus, fight formula)	All match the specification
Admin panel	Headless browser through every page, desktop and phone width	No console errors; layout verified
Email failure handling	Wrong Gmail credentials	Incident saved; email marked Failed with the SMTP error; no crash

8.1 Performance (CPU only, AMD Ryzen 5 5625U, no GPU)

Stage	Time per call
Person YOLOv8n	68 ms
Weapon YOLO11n	81 ms
Fire/smoke YOLOv8n	90 ms
Violence ViT-B/16 (only with 2+ people)	230 ms
YOLO-World S (blood)	197 ms
CLIP ViT-L/14 (blood + cross-checks, every 3 s)	1,165 ms
Overall	**Video stream about 15 fps; detection 2.5 to 5 analyses per second**

Model loading takes about 25 seconds at start-up. With an NVIDIA GPU (CUDA build of PyTorch), detection is roughly ten times faster.

9. Installation and running

The full step-by-step guide is in `SETUP.md` in the repository. In short:

9.1 Requirements

Software	Notes
Windows 10/11 laptop with a webcam	8 GB RAM, about 5 GB free disk; internet for the first setup
Python 3.12 (3.10-3.12)	Tick 'Add python.exe to PATH' during installation
Node.js LTS	nodejs.org
Git	git-scm.com
MongoDB Community Server	Installed as a Windows service (or a MongoDB Atlas connection string)

9.2 Steps

```
git clone https://github.com/road2tec/ai-surveillance-system.git
cd ai-surveillance-system
setup.bat          :: installs everything, creates .env files, downloads models
notepad server\.env :: set SMTP_USER, SMTP_PASS (Gmail App Password), ALERT_TO,
                    ::     ADMIN_EMAIL, ADMIN_PASSWORD
start.bat          :: starts the three services and opens the browser
```

Open **http://localhost:5173**, sign in with ADMIN_EMAIL / ADMIN_PASSWORD, wait for the models to load, and click **Start camera**.

9.3 Gmail App Password

- Sign in to the Gmail account that sends alerts and enable 2-Step Verification (myaccount.google.com/security).
- Open myaccount.google.com/apppasswords, create a password named 'Surveillance' and copy the 16 characters.
- Put the address in SMTP_USER and the App Password in SMTP_PASS in server\.env, restart, then use **Send test email** on the Settings page.

9.4 Configuration reference

File	Key settings
server/.env	PORT, MONGO_URI, JWT_SECRET, INGEST_API_KEY, AI_SERVICE_URL, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, ALERT_TO, ADMIN_EMAIL, ADMIN_PASSWORD, CLIENT_URL
ai-service/.env	BACKEND_URL, INGEST_API_KEY (same as server), CAMERA_INDEX (0 = webcam, or a video path / RTSP URL), CAMERA_NAME, DEVICE (auto / cpu / cuda)
client/.env	VITE_API_URL, VITE_SOCKET_URL
ai-service/app/config.py	Model candidates, class mappings and every threshold

10. Limitations and future work

10.1 Limitations

- The models are general public checkpoints, not tuned to this camera or room, so some misses and false positives are expected. Examples from testing: the fire model does not box large smoke plumes, and the weapon model occasionally labels an ornate object as a gun at low confidence.
- The violence model classifies single frames. Its raw output is unreliable in low light, which is why it is combined with a person count and a CLIP check. It also scored a staged boxing photo low.
- Blood detection is zero-shot and should be treated as an indication, not evidence.
- On a CPU-only laptop, detection runs a few times per second. A GPU is recommended for more cameras.

10.2 Future work

- Fine-tune the weapon, fire and violence models on footage from the deployment site.
- Use a video-clip model (e.g. VideoMAE) for violence, to use motion instead of single frames.
- Support several cameras with a camera management page.
- Add SMS / WhatsApp / push notifications and an escalation policy.
- Add object tracking to follow a weapon carrier across frames.
- Package the services as Docker containers for one-command deployment.

10.3 Conclusion

The project delivers a working end-to-end system. A live camera feed is analysed by several pretrained detectors, the results are fused into an interpretable threat score, and confirmed threats are stored, emailed with evidence and pushed to a live admin panel within seconds. Every model was checked before use, real problems found during testing were fixed with documented safeguards, and the whole system installs on a new Windows laptop with a single setup script.

Appendix: project structure

```
ai-surveillance-system/
  ai-service/            Python FastAPI detection service
    app/main.py          endpoints, MJPEG stream
    app/camera.py        capture / detect / blood threads, annotation
    app/detectors/       person, weapon, fire_smoke, fight, blood
    app/scoring.py       threat score, levels, cooldown
    app/reporter.py      queued delivery to the backend
    app/model_hub.py     Hugging Face verification and download
    app/config.py        model ids, class maps, thresholds
    download_models.py   pre-download + model report
    test_detectors.py    offline check on images
  server/                Express API (models, routes, middleware, mailer, seed)
  client/                React admin panel (pages, components, context)
  docs/                  this document
  setup.bat, start.bat   one-click install and start
  README.md, SETUP.md    technical reference and installation guide
```